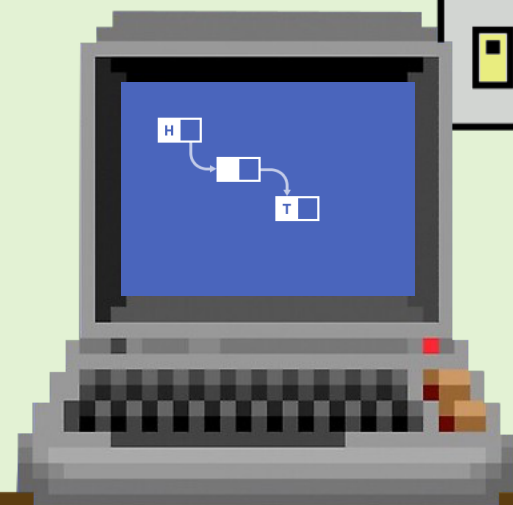


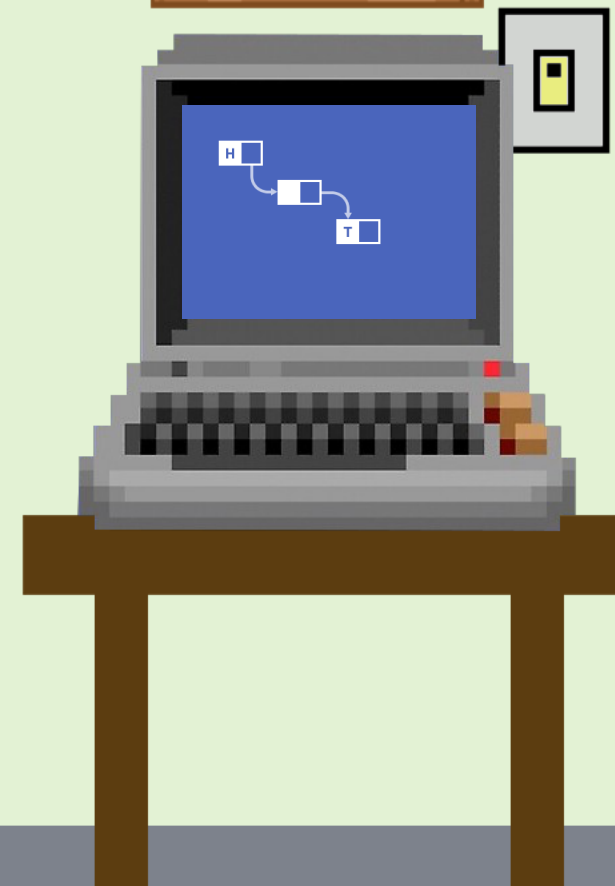
Listas

Programação Funcional



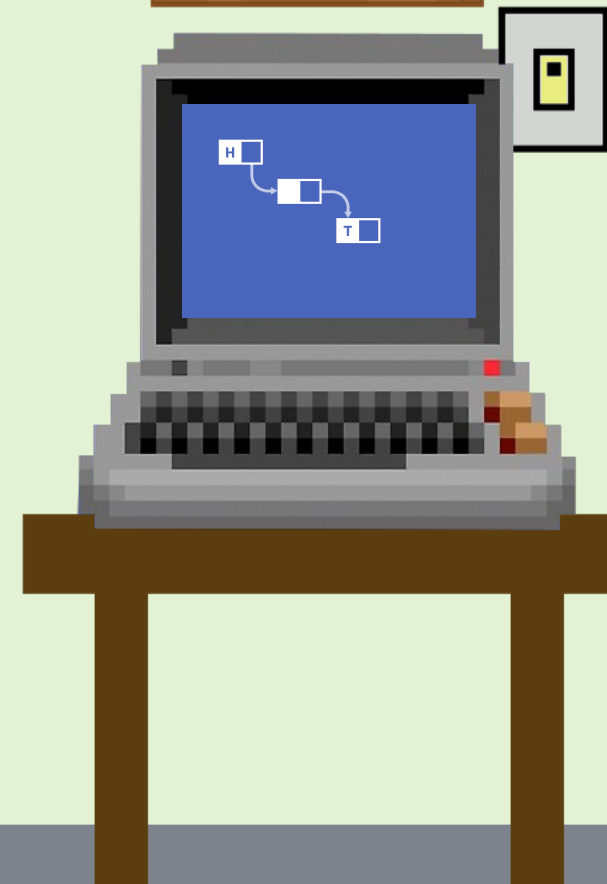
Listas

- Recurso fundamental nas linguagens funcionais
- Coleção homogênea de dados sem limite de tamanho
- Geralmente é um tipo primitivo
 - Imensa gama de funções para sua manipulação
- A notação utilizada para as listas é colocar seus elementos entre colchetes
 - [1,2,3,4,1,3] Lista de inteiros
 - [True, False] Lista de booleanos
 - ["Marta", "Marlene"] Lista de Strings



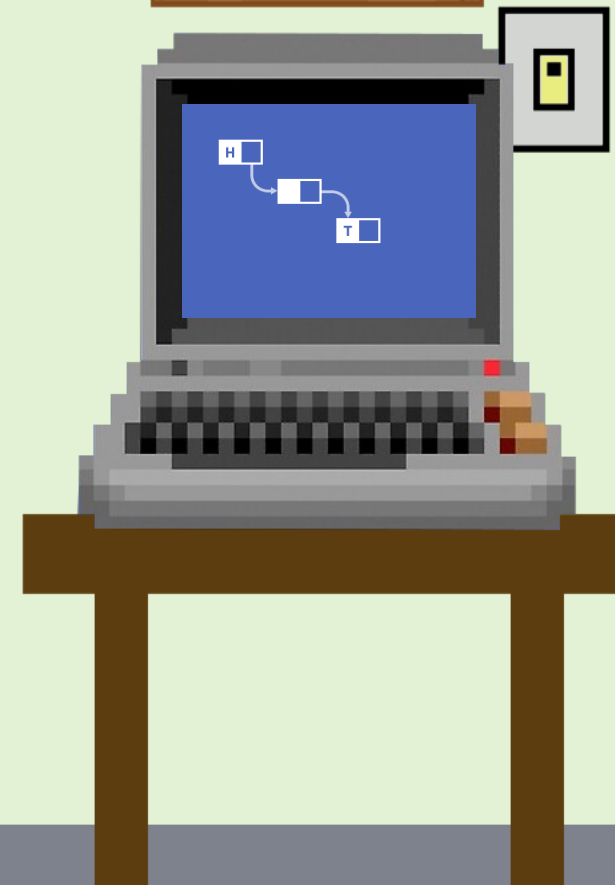
Listas

- Haskell só admite listas homogêneas
 - `[False, 2, "Maria"]` Não é uma lista válida
- Atenção
 - `[dobro, triplo]` Lista de funções `(Int->Int)`
 - `[[12,1], [3,4], [4,4,4,4,4], [ ]]` Lista de `[Int]`
- Lista vazia, simbolizada por `[ ]`
 - Pode ser de qualquer tipo
- Lista não vazia, simbolizada por `(a : x)`
 - `a` representa um elemento da lista (possui um tipo)
 - `x` representa outra lista de elementos do tipo de `a`
 - O elemento `a` é chamado de cabeça e `x` de cauda



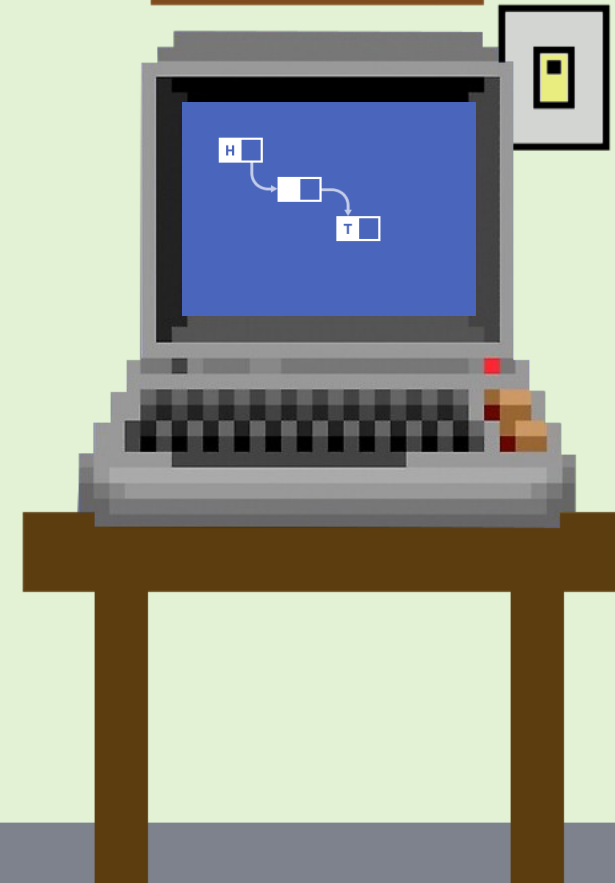
Propriedades

- A ordem em uma lista é importante
 - $[1,3] \neq [3,1]$
 - $[\text{False}] \neq [\text{False}, \text{False}]$
- A lista $[m .. n]$ é igual à lista $[m, m+1, \dots, n]$
 - $[1..5] = [1,2,3,4,5]$
 - $[3.1 .. 7.0] = [3.1, 4.1, 5.1, 6.1, 7.1]$
- A lista $[m, p .. n]$ é a lista de m até n com passo $p-m$
 - $[7,6..3] = [7,6,5,4,3]$
 - $[0.0, 0.3 .. 1.0] = [0.0, 0.3, 0.6, 0.9]$



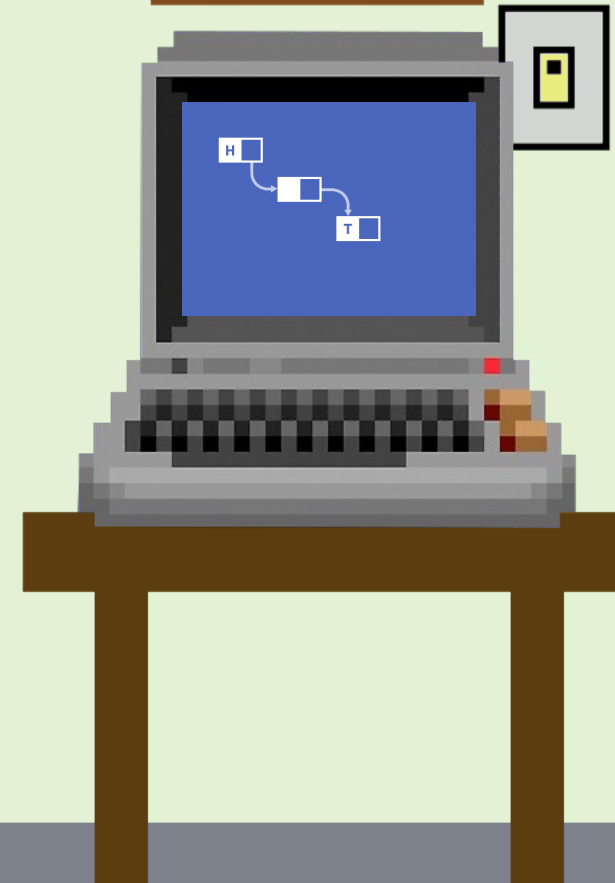
Propriedades

- A lista $[m..n]$, para $m > n$, é vazia
 - $[7..3] = []$
- Listas vazias não tem cabeça e nem cauda
- Listas **não** vazias tem cabeça e cauda
 - A cauda é também uma lista (pode ser vazia)
 - $[7] = (7 : [])$



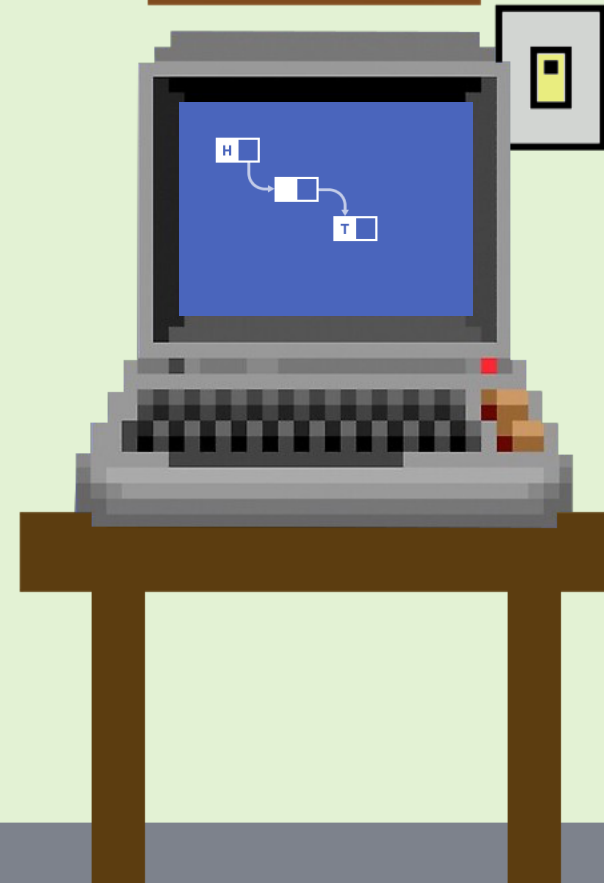
Construtor

- Chamado de cons é sinalizado por : (dois pontos)
- Operador que toma como argumentos um elemento de um tipo e uma lista de elementos do mesmo tipo
 - Insere o elemento como a cabeça da nova lista
 - Associa pela direita
- Exemplos
 - $10 : [] = [10]$
 - $2 : 1 : 3 : [] = 2 : 1 : [3] = 2 : [1,3] = [2,1,3]$



Funções

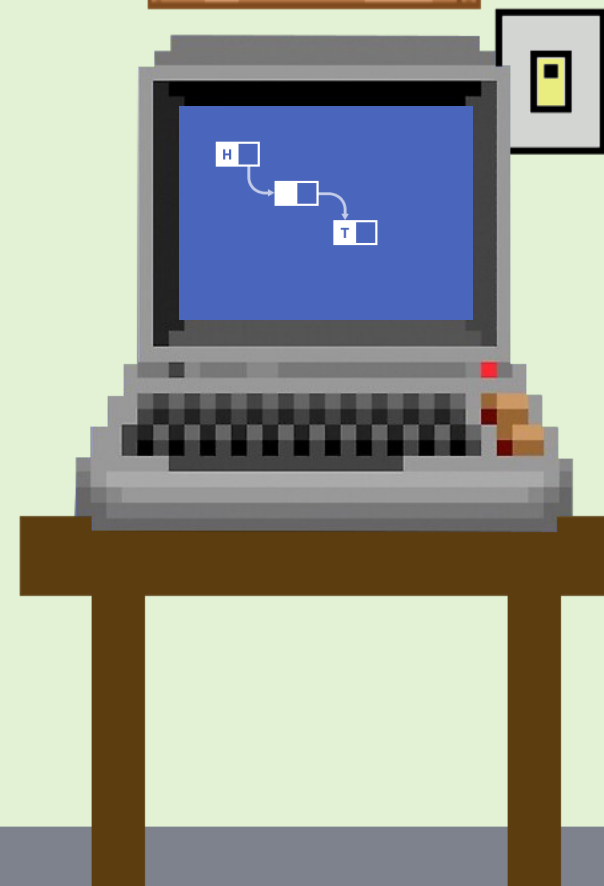
- `null :: [t] -> Bool`
- `head :: [t] -> t`
- `tail :: [t] -> [t]`
- `reverse :: [t] -> [t]`
- `length :: [t] -> Int`
- `sum :: [t] -> Int`
- `product :: [t] -> Int`
- `take :: Int -> [t] -> [t]`
- `drop :: Int -> [t] -> [t]`
- `!! :: [t] -> Int -> t`
- `++ :: [t] -> [t] -> [t]`



Exemplo

- Criar a função somaLista (recebe uma lista de inteiros e retorna a soma de todos os elementos dessa lista)
 - Definir cabeçalho da função
 - somaLista:[Int]->Int
 - Definir caso base
 - somaLista [] = 0
 - Definir caso recursivo
 - somaLista (x:xs) = x + somaLista xs

```
somaLista:[Int]->Int  
somaLista [] = 0  
somaLista (x:xs) = x + somaLista xs
```



Exemplo

```
somaLista::[Int]->Int  
somaLista [] = 0  
somaLista (x:xs) = x + somaLista xs
```

```
somaLista [5,9,2]  
=somaLista (5:[9,2])  
=5 + somaLista [9,2]  
=5 + somaLista (9:[2])  
=5 + 9 + somaLista [2]  
=5 + 9 + somaLista (2:[])  
=5 + 9 + 2 + somaLista []  
=5 + 9 + 2 + 0  
=16
```



Exercício

- Criar a função `replicaChar`
 - Recebe um `char` e um valor inteiro `n`
 - Cria uma lista de caracteres formado pelo caractere replicado `n` vezes

```
replicaChar::Char->Int->[Char]
replicaChar c n
  | n==0 = []
  | otherwise = c:replicaChar c (n-1)
```



Exemplo

```
replicaChar :: Char -> Int -> [Char]
replicaChar c n
  | n == 0 = []
  | otherwise = c : replicaChar c (n-1)
```

```
replicaChar 'x' 3
='x' : replicaChar 'x' (3-1)
='x' : replicaChar 'x' 2
='x' : 'x' : replicaChar 'x' (2-1)
='x' : 'x' : replicaChar 'x' 1
='x' : 'x' : 'x' : replicaChar 'x' (1-1)
='x' : 'x' : 'x' : replicaChar 'x' 0
='x' : 'x' : 'x' : []
=['x','x','x']
```



Exercício

- Definir função `inverte`
 - Recebe uma lista de caracteres
 - Inverte a ordem dos elementos da lista

```
inverte:[Char]->[Char]
inverte [] = []
inverte (a:as) = inverte as ++ (a:[])
```



Exemplo

```
inverte:[Char]->[Char]  
inverte [] = []  
inverte (a:as) = inverte as ++ (a:[])
```

```
inverte [3,2,1]  
=inverte (3:[2,1])  
=inverte [2,1] ++ (3:[])  
=inverte (2:[1]) ++ [3]  
=inverte [1] ++ (2:[]) ++ [3]  
=inverte (1:[]) ++ [2] ++ [3]  
=[] ++ [1] ++ [2] ++ [3]  
=[1,2,3]
```



Exercício

- Ordenar lista de inteiros
 - Método: Construir uma nova lista a partir da lista inicial, mas posicionando as cabeças corretamente

```
ordena:[Int]->[Int]
ordena [] = []
ordena (x:xs) = insere x (ordena xs)
```

```
insere::Int->[Int]->[Int]
insere x [] = x:[]
insere x (c:cs)
  | x <= c = x:c:cs
  | otherwise = c:(insere x cs)
```



Exemplo

```
ordena::[Int]->[Int]
ordena [] = []
ordena (x:xs) = insere x (ordena xs)
```

```
insere::Int->[Int]->[Int]
insere x [] = x:[]
insere x (c:cs)
  | x <= c = x:c:cs
  | otherwise = c:(insere x cs)
```

```
ordena [5,9,2]
=ordena (5:[9,2])
=insere 5 ordena [9,2]
=insere 5 ordena (9:[2])
=insere 5 (insere 9 ordena [2])
=insere 5 (insere 9 ordena (2:[]))
=insere 5 (insere 9 (insere 2 ordena []))
=insere 5 (insere 9 (insere 2 []))
=insere 5 (insere 9 2:[])
=insere 5 (2:(insere 9 []))
=insere 5 2:[9]
=2:(insere 5 [9])
=2:5:[9]
=2:[5,9]
=[2,5,9]
```

